

OpenAI Codex: Operational Reference

OpenAI Codex (CLI) — Operational Reference

A no-fluff, implementation-focused breakdown. Each item lists **what / where / why**.

1. Core Operating Premises

- **Layered config** — global user defaults + project overrides; project config only loads when the project is **trusted**.
- **Instruction chain** — Codex assembles all AGENTS.md files into one instruction set at startup.
- **Safety by default** — starts in the most restrictive approval + sandbox mode; escalate deliberately.
- **Reusable context** — store prompts, profiles, and MCP servers in config rather than retyping.

2. The ~/.codex/ Directory Layout

Path	Scope	Purpose
~/.codex/ (macOS/Linux)	Global	Config, credentials, session history, logs
%USERPROFILE%\ .codex\ (Windows)	Global	Same as above
\$CODEX_HOME	Override	Env var to relocate the config dir
~/.codex/config.toml	Global	Personal defaults for all projects
~/.codex/AGENTS.md	Global	Persistent instructions, every session
~/.codex/prompts/	Global	Reusable custom prompt templates
<project>/ .codex/config.toml	Project	Overrides global config
<project>/ .codex/AGENTS.md	Project	Project-specific instructions
<project>/AGENTS.md (+ nested)	Project	Hierarchical, per-directory instructions (monorepos)

3. AGENTS.md (Instruction Files)

- **Purpose:** Tells Codex how to behave; auto-loaded before any task.
- **Precedence (low → high):** global → project root → nested directory.
- **Use cases:**
 - Root: stack, conventions, build/test commands.
 - Service/package level: local rules in monorepos.
- **Premise:** Nearest file in the tree wins for overlapping rules.

4. config.toml (Behavior Config)

- **Location:** ~/.codex/config.toml (global) and <project>/ .codex/config.toml (project).
- **Format:** TOML (human-readable).
- **Controls:** model params, API connectivity, execution safety, sandbox, caching, MCP servers, profiles.
- **Override per-invocation:** --config key=value.
- **Security:** Never store API keys here — use env vars or system keychain.
- **Pattern:** Maintain separate files for dev vs prod contexts.

5. Approval Modes

Mode	Behavior
suggest (default)	Approve every action; shows diffs; waits before writing/running
auto-edit	File edits auto-approved; commands may still need approval/sandbox
full-auto	Full autonomy + system access; not for production/sensitive data
• Switch mid-session: /mode. • Flag: --ask-for-approval → untrusted on-request never.	

6. Sandbox Modes

Mode	Behavior
none	No sandbox, full system access
read-only	Read files only; no writes/network
workspace-write	Write to workspace; configurable network
docker	Commands isolated in a Docker container
danger-full-access	Full access — containers only
• Bypass everything: --dangerously-bypass-approvals-and-sandbox (alias --yolo) — only in externally hardened environments.	

7. Custom Prompts

- **Location:** ~/.codex/prompts/
- **Purpose:** Reusable templates for common tasks.
- **Invoke:** /prompts: slash menu.

8. MCP (Model Context Protocol)

- **Configured in:** config.toml (MCP server entries).
- **Manage:** list / add / remove / authenticate servers.
- **Inverse mode:** Codex can itself run **as** an MCP server.
- **Premise:** Add only needed servers — extra tools dilute decisions.

9. Key Commands & Flags

Slash commands

Command	Function
/mode	Switch approval mode
/permissions	Change what Codex can do
/sandbox-add-read-dir	Grant read access to a dir
/agent	Switch active agent thread
/model	Select model
/diff	Show Git diff
/clear	Clear terminal

CLI flags

Flag	Function
--model	Override model
--cd	Set working directory
--add-dir	Grant extra write access
--image	Attach image files
--config key=value	Override a config value
--ask-for-approval	Set approval policy
--yolo	Bypass approvals + sandbox

10. Profiles

- **Defined in:** config.toml.
- **Purpose:** Named bundles of settings (model, approval, sandbox) for different contexts.
- **Use:** Swap whole configurations per task/environment.

11. High-Leverage Habits

- Start restrictive (suggest + read-only), escalate only when needed.
- Keep AGENTS.md lean; push local rules to nested files.
- Use profiles for dev/prod separation.
- Keep secrets out of config.toml.
- Prefer docker/workspace-write over full access for autonomy.